Advances in Cognitive Systems X (2026) 1–20 Submitted 8/2026; published 10/2026

# A Unified Account of Concepts and Chunks

**Karthik Singaravadivelan** KSINGARA3@GATECH.EDU
**Pat Langley** PATRICK.W.LANGLEY@GMAIL.COM
Georgia Institute of Technology, Atlanta, GA 30332 USA
Institute for the Study of Learning and Expertise, Palo Alto, CA 94306 USA

## Abstract

Cognitive psychology has studied how people encode, use, and learn concepts that describe categories, and how they represent, recognize, and acquire chunks for familiar patterns of elements. The literatures on these two topics are nearly disjoint, which poses a challenge for unified theories of cognition. In this paper, we review Cobweb, a computational account of categorization and concept formation, then propose an extended theory that incorporates chunks and their acquisition. The theory makes no commitments about modality, applying to any experience that decomposes into elements and relations among them. We also present TRELLIS, an implementation of this theory, and illustrate its application to learning context-free grammars, which we adopt as a testbed because they involve both concept-like and chunk-like elements. In addition, we report experimental results on learning for three synthetic grammars that demonstrate the system's ability to represent syntactic knowledge, use it to parse and generate sentences, and learn compositional structures from sample parses. We conclude by discussing related work on concepts and chunks, along with directions for future research on the problem.

## 1. Introduction

A primary aim of science is to develop general theories that explain as broad a range of phenomena as possible. Within cognitive systems, such theories involve postulates about the mental structures stored in memory, the performance processes that operate over them, and the learning mechanisms that acquire them. Different paradigms take differing positions on these issues, but all share a concern with generality, seeking broad coverage from a small set of assumptions.

In this paper, we propose a unified computational account of two facets of cognition that researchers have long treated as distinct and disconnected. One of these revolves around *concepts*, which Langley (2025) defines as:

- *A concept is a cognitive structure that denotes a set of entities or situations and that characterizes them in terms of their regularities*.

The representation, use, and learning of concepts has been a major topic in cognitive psychology (Rips et al., 2012) and artificial intelligence (Langley, 1996) since the 1960s. A second area, less popular but one that still receives substantial attention, concerns *chunks*, which Langley defines as:

- *A chunk is a cognitive structure that denotes a collection of entities, possibly typed, that form a specified configuration or relational pattern*.

© 2026 Cognitive Systems Foundation. All rights reserved.

Treatments of chunk encoding, recognition, and acquisition have mainly appeared in the psychological literature (Miller, 1956; Fountain & Doyle, 2012), but there has been some AI work (Campbell & Berliner, 1983). Topics like concept generalization and similarity have been recurring themes in work on concepts, whereas research on chunks has emphasized the composition of higher-level structures from their constituents.

Concepts and chunks have much in common, but work on the former has emphasized generalization, whereas studies of the latter have focused on composition. Both capabilities are necessary for robust cognition, but neither alone is sufficient, which suggests that most existing theories are incomplete. A system that supports generalization without composition is limited to encoding knowledge at a single level, whereas one that acquires configurations without generalizing them cannot use them in novel situations. Both capacities are needed. Composable chunks supply parts and relations among them, whereas generalized concepts extend their coverage.

In the remaining pages, we present a unified theory that addresses both aspects of cognition. Our approach builds on Cobweb (Fisher, 1987), a computational account of concept representation, organization, retrieval, and formation. We start by reviewing Cobweb's main assumptions, which our framework retains, and then introduce postulates that support chunk-related phenomena. After this, we describe TRELLIS, an implementation of the theory, and report experimental results on grammar induction. We focus on this task because syntax acquisition involves both generalization and composition, but the theory should apply equally well to vision, problem solving, and other cognitive functions. In closing, we examine related ideas and note avenues for future research.

## 2. The Cobweb Framework

Fisher's (1987) Cobweb was an innovative model of categorization and concept formation that combined ideas from decision-tree induction, Bayesian classifiers, and nearest-neighbor methods. The system received considerable attention within the maturing machine learning community and made contact with important psychological findings like basic-level and typicality effects. Excitement about the approach led to multiple extensions, many of them documented in Fisher, Pazzani, and Langley (1991). In this section, we review the framework's main theoretical tenets, which it shares with its descendants, including the new system we report later.

### 2.1 Representation in Cobweb

The Cobweb framework's most basic postulates concern how conceptual knowledge is represented in long-term memory, as later assumptions build directly upon them. These include statements that:

**R1.** Long-term memory contains two types of structures: instances and concepts.

**R2.** An instance is a set of attribute-value pairs that describe a single observation.

**R3.** A concept specifies, for each attribute, a probability distribution over that attribute's values.

That is, we can view each concept as a probabilistic generalization for a set of instances in which each attribute's distribution is independent given the concept. Unlike many other theories of categorization, Cobweb retains both types of structures in memory and we can view each instance as a very specific concept with a heavily skewed distribution.

### 2.2 Organization in Cobweb

Of course, the contents of long-term memory must be organized in some fashion, so the Cobweb framework also takes strong positions about their arrangement. These include postulates that:

**O1.** Concepts reside in a taxonomic hierarchy with a single root and instances as terminal nodes.

**O2.** Each nonterminal node is a probabilistic summary of the instances below it in the taxonomy.

**O3.** Each nonterminal node has two or more children that partition the instances it summarizes.

This organization resembles that produced by hierarchical clustering systems, except that each nonterminal node (cluster) in the taxonomy has a concept description rather than a set of instances, which are stored at terminal nodes.

The taxonomy is also similar to a decision tree, except that it stores 'tests' on nodes rather than links. Also, the tests are multivariate in that each child describes distributions over a set of attributes rather than just one. Like a decision tree, a Cobweb hierarchy partitions instances into mutually exclusive sets, but each node is a probabilistic generalization of instances rather than a logical characterization. We can also view paths through the taxonomy as *indexing* nodes in long-term memory, which has implications for performance.

### 2.3 Performance in Cobweb

Claims about representation and organization are not enough for a complete theory. Cobweb also posits performance mechanisms that use its concepts, and their arrangement in long-term memory, to produce behavior. We can summarize the framework's assumptions on this front as:

**P1.** There are two linked performance mechanisms: *classification* and *prediction*.

The first assigns a case to a category, whereas the second infers any values for missing attributes. This is similar to the distinction in the psychological literature between recognition and recall.

**P2.** Classification involves sorting an instance downward through the taxonomy.

Thus, classification is much like that in decision trees (Quinlan, 1986), except that Cobweb selects the best child at each level. The criterion it uses is *category utility* (Fisher, 1987), a probabilistic measure that favors children with higher intra-category similarity and lower between-category similarity. The process steps greedily down the tree, selecting the best option at each level.[1]

**P3.** Sorting continues until reaching a terminal node or no child is better than the current node.

This is another important difference from decision trees, which always sort cases to terminal nodes. In contrast, Cobweb halts at a nonterminal node when none of its children have a better score.

**P4.** At the halting node, prediction imputes the most probable values for unknown attributes.

Again, this scheme resembles that in decision trees, but rather than predicting a class label, Cobweb infers values for all missing attributes, an ability that Fisher (1987) called *flexible prediction*.

1. Some variants, like Cobweb/4V (Barari et al., 2024), replace this greedy descent with best-first search, but we will adopt the more basic scheme here, as in the original version of Cobweb.

### 2.4 Learning in Cobweb

Finally, the theory makes statements about how conceptual knowledge is acquired through learning. These additional postulates include:

**L1.** Learning is an incremental, online process that is fully interleaved with performance.

**L2.** Learning is unsupervised in that instances do not come with class labels.

That is, sorting each new instance changes both the content of existing concepts and their organization in long-term memory. Moreover, the learner must invent its own categories rather than obtain them from an outside source. Training cases can include a 'class' attribute, but this feature is treated no differently from others.

Furthermore, the Cobweb theory posits two varieties of incremental learning, one statistical in character and the other structural in nature:

**L3.** Sorting an instance through a node in the taxonomy updates its associated probabilities.

This mechanism simply alter counts that reflect the prior probability of a concept given its parent and the probability of each attribute value given that concept, much as in naive Bayesian classifiers. These changes are determined entirely by the instance's content and the path it traverses.

**L4.** At each node, sorting considers four structural options: adding the instance to an existing child, creating a new child, splitting a child, or merging two children.

The first option does not revise taxonomic structure; it simply updates probabilities for the child. The second alternative creates a new child based on the instance. The latter two actions lead to local restructuring before sorting continues. The same evaluation criterion used in routing determines which of these steps to take. Because Cobweb learns incrementally, the order in which it encounters instances can influence the structure of the acquired taxonomy. Merging and splitting can mitigate these order effects, resulting in hierarchies with better organizations.

The framework also invokes a deterministic restructuring action when an instance reaches a terminal node N. In these cases, the learning process creates two new terminal nodes, one based on the stored instance and the other based on the new case. The existing node N becomes their parents, with its probabilities updated based on values in the recent instance. Together, these operations support the incremental and unsupervised acquisition of a taxonomy of probabilistic concepts, which are key characteristics of human category learning.

## 3. A Unified Theory of Chunking and Categorization

Cobweb unifies ideas from decision trees, naive Bayesian classifiers, and nearest neighbor in an elegant framework, and it makes contact with the psychology literature on categorization and concept learning. However, it does not explain the representation, organization, use, or acquisition of chunks, which are equally important facets of human and machine cognition. The issue is that Cobweb has nothing to say about *composition*, which is central to chunks and their processing. This section introduces new postulates that, when combined with those just described, provide an extended theory that links concepts and chunks. We use examples from language syntax to clarify the ideas because they require both capabilities, although the framework has much broader implications.

### 3.1 New Representation Postulates

The extended theory retains Cobweb's representational assumptions, including the distinction between instances and concepts, but adds postulates that make it richer. These include:

**R4.** An *experience* is a set of elements and the local relations among them.

For example, someone reading text encounters a set of words that are arranged in a linear sequence. The words are elements that are related by a *before* or a *left-of* relation.

**R5.** Every element is either a *primitive* structure or a *composite* structure.

Primitive elements are the basic building blocks of memory, similar to instances and concepts in Cobweb. A composite, in contrast, is an element built from primitives or lower-level composites, which its content description specifies. Composites correspond to *chunks*.

**R6.** Each element can be described by its *content* or its *context*.

The first of these specifies an element's structure in terms of its constituents (e.g., *black cat*). The second specifies other elements in its vicinity (e.g., *the*, *small*, *chased*), along with their relations to the element (e.g., *before* or *after*). We will refer to these as *content instances* and *context instances*.

### 3.2 New Organization Postulates

The expanded theory also retains Cobweb's commitment to organizing concepts in taxonomies, but structures composed of others require further assumptions. Taken together, the content fields of composites impose an implicit *partonomic* hierarchy, and the distinction between content and context instances leads to additional organizational postulates. These include:

**O4.** Long-term memory contains a *content taxonomy* that organizes concepts by their contents.

For primitive elements, this hierarchy corresponds to that in Cobweb, which clusters instances that have similar attribute values. However, for composite elements, it groups cases that have similar sets of constituents. For example, the chunks *black dog* and *black cat* share the element *black*.

**O5.** Long-term memory contains a *context taxonomy* that organizes concepts by their contexts.

This hierarchy instead organizes concepts that occur in similar contexts. For instance, it might group the words *dog* and *cat* together because they appear in like surroundings. The same applies to composite structures, like different types of noun phrases, which occur both before and after verbs. These two taxonomies are intertwined in that nodes in one point to nodes in the other. However, keeping them separate acknowledges that one can organize memory either in terms of content (i.e., the constituents of chunks) or in terms of context (i.e, the situations in which chunks occur).

### 3.3 New Performance Postulates

Performance in Cobweb focuses on classification and prediction, but the introduction of chunks supports new types of cognitive activities. In language processing, these are referred to *parsing* and *generation*, and the extended theory makes commitments to their operation as well.

**P5.** Parsing iteratively creates a partonomic tree from the bottom up, adding chunks that elaborate an input experience.

In language processing, this involves creating a parse tree from the words in a sentence by introducing nonterminal symbols like *noun*, *noun phrase*, and *verb phrase*.

**P6.** Parsing sorts candidates through both taxonomies and selects the best recognized chunk.

This requires a notion absent from Cobweb, a *recognition threshold* that a candidate must exceed at some node in the taxonomy before counting as familiar enough to proceed.[2] Among the candidates that pass, the best-scoring chunk joins the growing parse tree.

**P7.** Parsing halts when no candidate is recognized or when only one composite element remains.

The content score asks whether the composition looks like one used before; the context score asks whether the surroundings look like ones that have hosted similar composites. The termination condition allows graceful failure: when nothing clears the threshold, parsing ends ends rather than asserting spurious chunks at the boundary of competence.

The generation process also moves beyond Cobweb's capabilities. This proceeds in the opposite direction from parsing with the aim of creating a well-formed collection of new elements.

**P8.** Generation iteratively expands a composite element from the top down, decomposing chunks to produce an experience.

This is the inverse of parsing. Rather than combining elements into larger structures, it starts from a single composite and rewrites it into constituents, then rewrites those in turn until only primitives remain. For language, expanding a *sentence* into a *noun phrase* and *verb phrase*, decomposing them, and so forth, yields a grammatical sequence of words.

**P9.** Generation recalls candidate decompositions from the content taxonomy and conditions them on the context taxonomy.

This assumes a counterpart to the recognition threshold, which determines whether a candidate is familiar enough to use in parsing. Here we need a *recall threshold* that determines whether to use a content concept for an expansion (Gupta et al., 2025). If this is too low, then generation will draw on very specific nodes supported by only a few cases; if too high, it will favor general nodes that mix constituents which do not belong together. Recall samples from the most specific chunks that exceed the threshold to generate candidate expansions, then uses the context hierarchy to rank them.

**P10.** Generation terminates when every expanded element is a primitive.

The two taxonomies play complementary roles. The content hierarchy specifies candidate chunk expansions, whereas the context hierarchy specifies how well they fit their surroundings. For grammatical expertise, the first is analogous to individual rewrite rules, such as *VP => Verb NP*, and the second to knowledge about which rule to choose. Because generation recombines familiar parts in new ways, it can produce experiences the system has never seen yet are consistent with what it has learned, which for language means grammatical sentences beyond the training corpus.

2. This idea is similar to Fisher's (1989) notion of a prediction frontier, in which the taxonomy marks nodes to use when inferring values. However, our theory instead assumes a frontier of nodes that are marked as recognizable for purposes of creating chunks, not for making predictions.

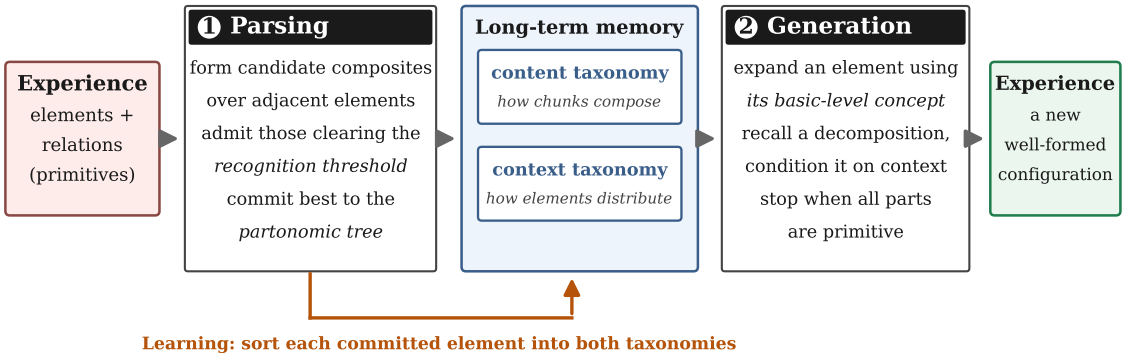



*Figure 1.* Modules of TRELLIS and the flow of an experience through them. Parsing composes primitive elements into a partonomic tree, consulting the content and context taxonomies that make up long-term memory, while generation inverts the process. Section 3 defines the terms set in italics.

### 3.4 New Learning Postulates

Learning in the extended theory does not require new postulates about mechanisms, as they remain unchanged from the Cobweb framework. However, it does make claims about the inputs to these already established processes. They include:

- Learning incorporates content and context instances extracted from elements into the content and context taxonomies, respectively.
- Learning incorporates instances for composite elements into both taxonomies, but incorporates instances for primitive elements only into the context taxonomy.

Neither statement should be surprising, as both follow from two other commitments: (a) the distinction between content and context instances, and (b) interleaving learning with performance. There is no separate mechanism for creating chunks, since parsing already produces candidates, and learning simply treats them as data for updating the taxonomies.

This does not reflect a limitation of Cobweb's account of learning. If anything, it provides evidence for substantial generality. Cobweb was designed to learn from independent and identically distributed data, which sequential domains like language clearly violate. Yet combining its original learning methods with the distinction between content and context instances, it can cover types of phenomena for which its creator did not intend.

## 4. TRELLIS: An Implementation of the Theory

The extended theory offers an account that unifies concepts and chunks, but it is abstract. To make it operational, we have developed TRELLIS, which embeds the postulates in an implemented system that invokes Cobweb as a subroutine. In this section, we describe how TRELLIS elaborates the theory and illustrate its choices with examples from language processing. We delay covering details like formulas for the recognition score until Appendix B.

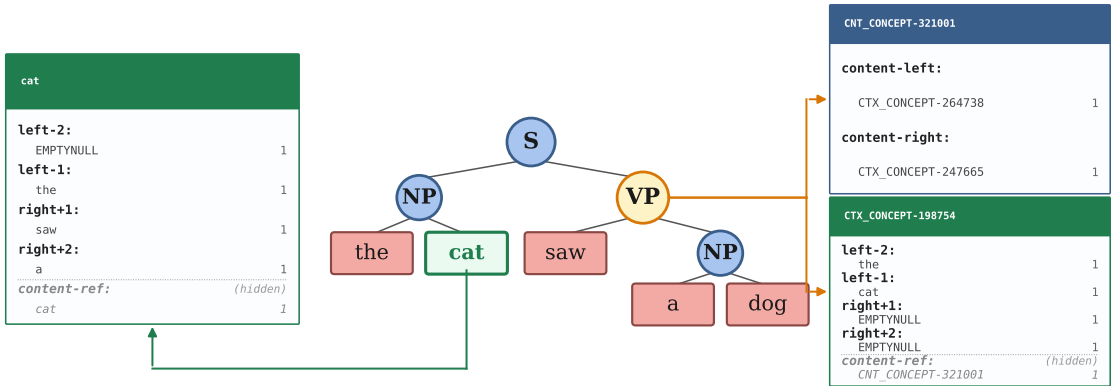



*Figure 2.* One primitive ("cat", green) and one composite ("saw a dog", amber) from a parse of "the cat saw a dog". Each box header is the long-term memory identifier to which the instance was sorted. The composite's content field points into the context hierarchy, keeping the two descriptions in complementary formats. Window width, distance weighting, and content-bag width are run-time values (Appendix B).

Figure 1 summarizes how the pieces fit together and clarifies our terminology. An experience enters as a set of primitive elements and relations among them. Parsing composes these into higher-level elements, admitting only candidates that clear a recognition threshold and committing the best to a partonomic tree. Long-term memory holds two taxonomies, over content and over context descriptions, which supply the scores that parsing consults. Generation runs the same machinery in reverse. Learning is not a separate stage: every committed element is sorted into both taxonomies, so the memory that scores later experiences is the one built from earlier experiences.

### 4.1 Representation in TRELLIS

Every element in TRELLIS includes two fields: *content* specifies the parts that compose the element, whereas *context* names its neighbors (MacLellan et al., 2022). Primitives have only a context field, while composites have both. Figure 2 illustrates one primitive and one composite drawn from the parse of a small sentence. A composite's content field encodes its left and right children with two attributes: an identifier that points to the context taxonomy (referring to the child itself) and a complexity tag (a small integer denoting its nesting depth). Both reside in the context hierarchy's identifier space rather than the surface vocabulary, so the content hierarchy never sees a primitive word. Every value encountered during a content sort was created by a context sort of the corresponding child. The complexity tag lets the content taxonomy identify composites whose children have similar contexts but differ in structural depth, such as NP=DET+N vs. S=NP+VP.

### 4.2 Organization in TRELLIS

Following the extended theory, TRELLIS divides long-term memory into two Cobweb taxonomies. The content hierarchy stores the content descriptions of composites and the context hierarchy stores the contexts in which they occur. They are linked in that a composite's content refers to identifiers

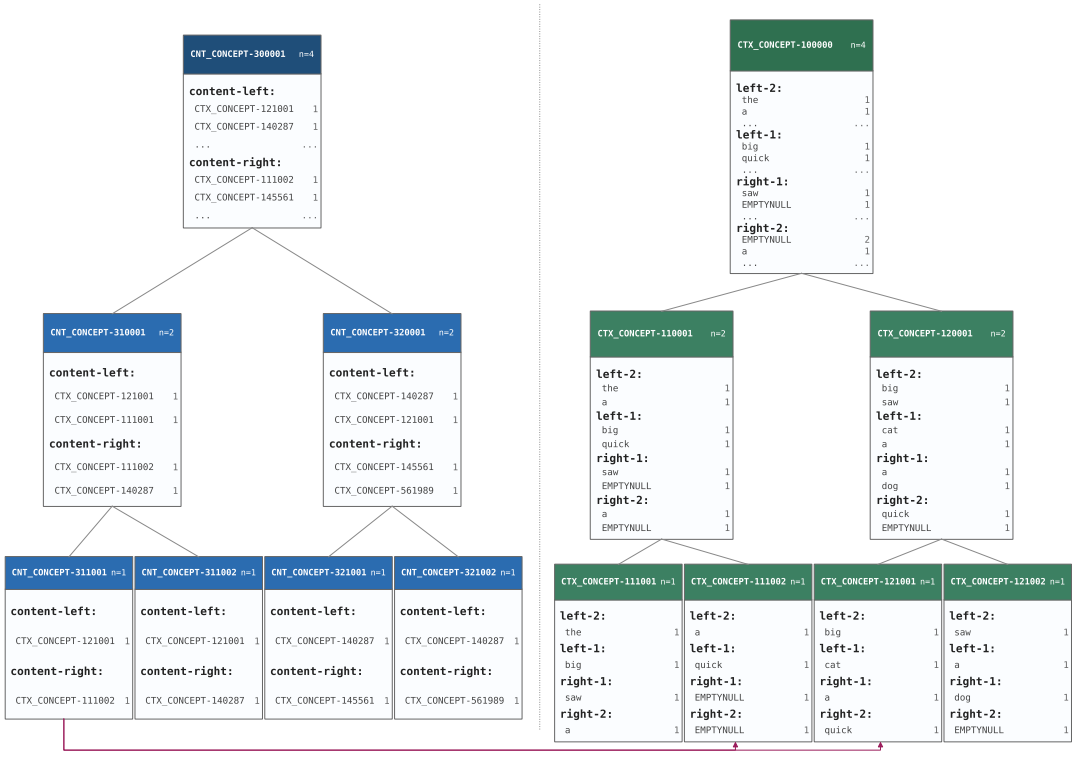



*Figure 3.* Leaf-level subtrees of the content (left) and context (right) hierarchies. Arrows show the cross-hierarchy link: a content leaf's child fields name the context concepts that describe its children.

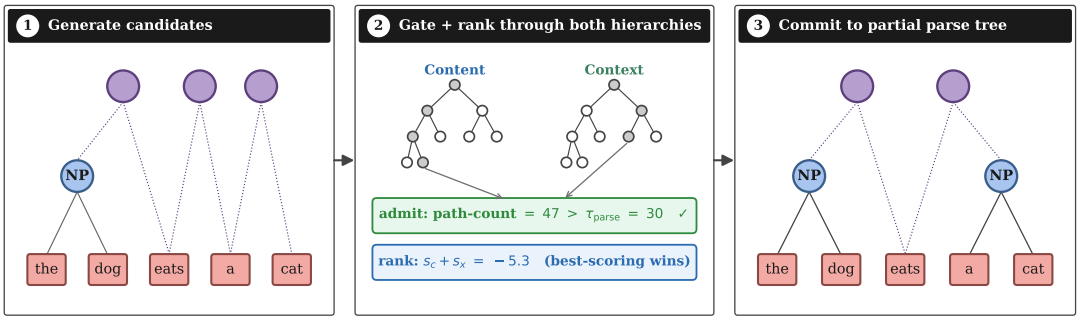
in the context hierarchy, so every description it encounters comes from an earlier context classification. Neither trades conceptual facets for compositional ones, since both structures carry each. Composites with structurally similar children appear near each other in the content tree, reflecting a shared role (say, as an NP-filling constituent), while a noun and an adjective phrase appear near each other in the context tree when they occur in comparable neighborhoods. Figure 3 shows leaf-level subtrees of each hierarchy and the cross-hierarchy references that link them.

### 4.3 Parsing and Generation in TRELLIS

The TRELLIS system includes two modules that expand on the theory's performance postulates. As Figure 4 (a) depicts, a parsing mechanism constructs parses from the bottom up by iteratively creating chunks that elaborate on the input experience. In addition, the generation mechanism shown in Figure 4 (b) constructs output experiences from the top down by iteratively decomposing chunks until its reaches primitive elements.

Parsing carries out greedy search through the space of possible parses. The module starts with a frontier of primitive elements, such as the words in a sentence, and repeatedly proposes candidate chunks over adjacent pairs. It sorts each candidate's content and context descriptions through their

(a) One parsing step




(b) One generation step

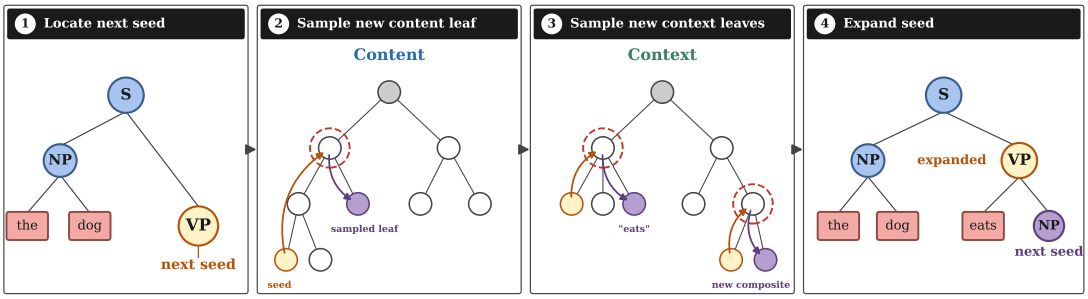



*Figure 4.* TRELLIS' two performance modules. (a) Parsing (1) forms candidate chunks over adjacent frontier elements; (2) sorts each through both taxonomies, admitting it iff some nonroot ancestor on the sort path has count above $\tau_{parse}$, ranking the admitted candidates by $s_c + s_x$; (3) commits the top-ranked one to the parse tree. (b) Generation (1) selects the next seed; (2) samples a fresh content leaf from a well-supported neighborhood above it; (3) resolves each of the sampled leaf's two child references against the class the parent expects; (4) expands the seed with these results. The numeric values in a2 are for illustration only.

respective hierarchies, scoring them by how well the candidate matches the concept at which sorting halts relative to its competitors (Appendix B). TRELLIS sums the two scores to produce a combined recognition score. For "the cat saw a dog", the first step would consider four adjacent word pairs.

TRELLIS admits a candidate only when a concept through which it was sorted has been visited enough times, with primitives gated on the context hierarchy and composites on the content hierarchy. The module walks upward from the node at which sorting halted; the candidate is recognized if the count of some nonroot ancestor exceeds the recognition threshold $\tau$. This gating scheme suppresses selection early in learning but has little effect later. After this, TRELLIS scores candidates by their posterior probabilities based on nodes in the two hierarchies and replaces the selected candidate's constituents with the composite element on the frontier. Parsing terminates if the module recognizes none of the candidate chunks, in which case it halts with a partial parse.

Generation proceeds in the opposite direction to produce a grammatical structure. This process begins from a seed node that is an unexpanded composite with an identifier in the content hierarchy.

The module samples from a *recombination pool* associated with the seed, decomposing the result into two children. To create the pool, TRELLIS walks up from the seed's leaf until it reaches an ancestor whose count exceeds the recall threshold $\tau$, analogous to the recognition threshold. At this point, it considers the constituent pairs stored at this node, each of which has an associated probability. Thus, the system draws from a set of decompositions it has seen often enough to treat as acceptable, from which it then samples, rather than ones with little empirical support. The sampling process can produce new combinations that no single training experience has contained.

After selecting a decomposition, TRELLIS uses the context hierarchy to filter alternatives. If a candidate's parent does not expect one of them, then the system eliminates it from competition to ensure it cannot emit an incorrect constituent. If the filter eliminates all candidates, say because the context concept has not yet acquired any compatible chunks, then the system falls back to a wider, less-filtered pool that incurs a small risk for errors of commission in return for broader coverage. Generation halts when every leaf of the parse tree is a primitive structure.

### 4.4 Learning in TRELLIS

Learning in TRELLIS does not require any new mechanisms beyond those in Cobweb, but it operates over what we have called 'experiences', which are collections of related elements. We provide system not only with training experiences, but also with desired parse trees for them. For example, the sentence *the big dog chased the black cat* would come with a parse into phrases at various levels. This constitutes a form of supervised learning, although nonterminal nodes in the parse trees are not themselves labeled with terms like *NP* and *VP*.

As with parsing, TRELLIS learns iteratively from the bottom up. This involves selecting an element on the frontier (initially the set of primitive structures) and sorting it through taxonomies. The system only sends primitive elements through the context hierarchy, as they have no internal structure. Composite elements are sent through both taxonomies, which incorporates them by their content and context descriptions. TRELLIS classifies composite elements only after it has already processed their constituents.

Once the system has handled every element of a parsed experience, it turns to the next, continuing until it exhausts the training data. Thus, learning occurs both incrementally within each experience and across a succession of experiences. TRELLIS has no separate routine for creating chunks: every composite structure becomes a new node in the taxonomies, and that node is what later parsing uses as the category label when the system meets a structurally similar chunk.

## 5. Evaluation of TRELLIS

We maintain that TRELLIS is a faithful implementation of our unified theory, but whether it operates as intended is another question. To answer it, we designed experiments on grammar acquisition that examined its ability to represent, organize, use, and learn linked content and context taxonomies. We describe the experimental design and test domains below, then present the results. As noted earlier, we focused on syntax because it requires both chunks and categories, but we anticipate our findings will carry over to other settings.

### 5.1 Experimental Design

The acquisition of grammatical expertise is harder to study than learning for classification tasks. We follow Langley and Stromsten's (2000) methodology, which recorded two dependent variables: *errors of omission* and *errors of commission*. The first measures how far a learned grammar is overly specific, in that it fails to parse or generate some legal sentences with correct parses. The second measures overgeneralization, in that the grammar parses or generates illegal constructions.

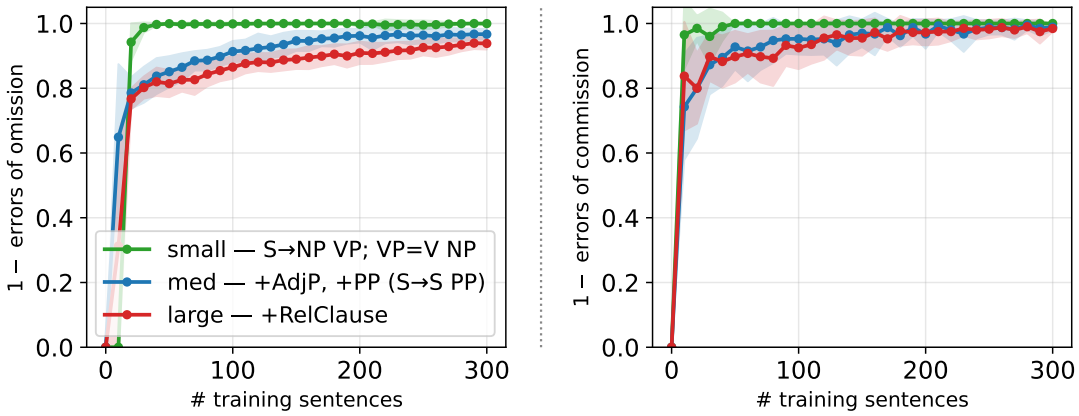
For our initial studies, we devised synthetic context-free grammars to generate training data, rather than a corpus of natural sentences like the Penn Treebank. To compute the two metrics, we used the target grammar to generate training sentences and associated parse trees, then let TRELLIS learn concepts and chunks from these experiences. To measure errors of omission, we used the target grammar to generate novel sentences, then used TRELLIS' parsing module to attempt parsing them. Similarly, to measure errors of commission, we called the generation module to produce sentences from the learned taxonomies, then used the target grammar to attempt parsing them. Because the system can acquire incomplete grammars, especially early in training, we did not use an all-or-none measure of success. Instead, we gave partial credit for each substructure of the target parse. In particular, we compared the *span* of words covered by each subtree,[3] as TRELLIS' memory has no class labels on nonterminals. For each test sentence, we took the fraction of substructures matched between the target and learned sentences.

The main purpose of our experiments was to reveal whether TRELLIS induces accurate grammatical knowledge or whether it forms overly specific or overly general structures. We were also interested in the rate of improvement as measured by learning curves. In addition, we also wanted to know how behavior scales in response to two types of grammatical complexity. One involved the number of nonterminal categories, such as *NP* and *VP*, in the target grammar. Raising this count increases the number of chunk types that the content hierarchy must encode and acquire. The other concerned the words per part-of-speech class. Increasing this number requires the context hierarchy to generalize across more surface forms.

In one experiment, we varied the number of nonterminal symbols, using three grammars with three, six, and eight nonterminals, which we will refer to as small, medium, and large, while holding number of words per class constant. In a second study, we varied the number of words per part of speech class from 11 (low) to 22 (medium) to high (39), while holding the number of nonterminals at six. Appendix A presents the rewrite rules and lexical items for each grammar, most of which involve recursion, while Appendix B provides the system parameters, which we did not vary. Each experimental condition used 400 sentence-parse pairs generated from the target grammar. To measure errors of omission, we used five-fold cross validation, holding out a different subset of 80 pairs for each fold in testing and the remaining 320 pairs in training. To measure errors of commission, we used the same five learned taxonomies to generate 80 sentence-parse pairs. Finally, we averaged each dependent variable over five runs based on the different training sets.

3. The grammar induction literature typically calls such spans *brackets*; we use *substructure* to avoid the Penn Treebank's labelled-bracket convention, which combines structural span agreement with part-of-speech or phrasal labels.

(a) Varying grammar complexity




(b) Varying lexicon size for the MED grammar

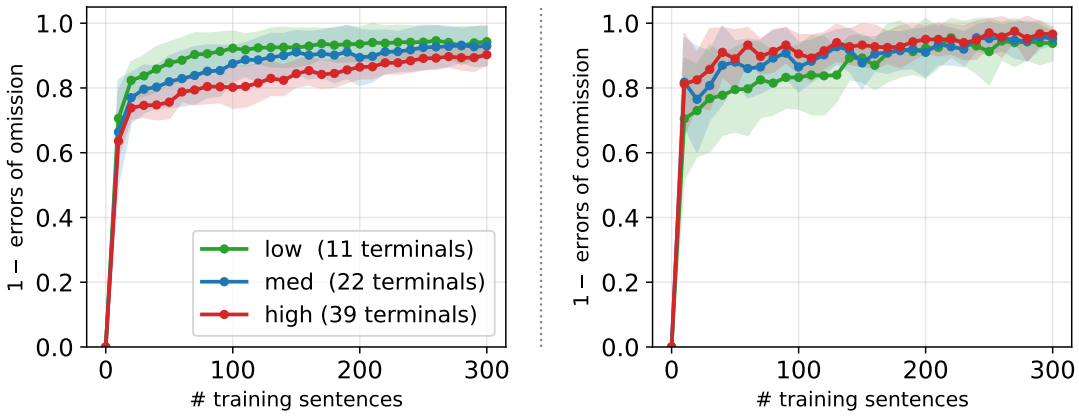



*Figure 5.* $1 -$ omission rate (parse, left of each pair) and $1 -$ commission rate (generation, right of each pair), across (a) the three grammars and (b) the three lexicon sizes (five data sets, $\pm 1\sigma$ band).

### 5.2 Experimental Results

Figure 5(a) shows the results of the first study, plotting one minus the omission rate on the left and one minus the commission rate on the right against the number of training sentences. Each graph shows three learning curves, one per grammatical complexity level. The most important result is that TRELLIS acquires grammatical expertise well, with both errors of omission and commission dropping to low percentages even for the most complex grammar, despite the inclusion of recursive rewrite rules in the targets. Learning is also rapid, with both errors dropping to 20 percent almost immediately and then improving gradually with more training sentences. Increasing the number of nonterminals slows learning, as the system requires examples of every chunk type to master the grammar, but the degradation is graceful. Inspection of the content hierarchy showed reasonable concepts for each nonterminal in the target grammar.

The plots in Figure 5(b) reveal similar results from the experiment that varied the lexicon size. As before, TRELLIS comes close to mastering the grammars from a moderate number of training sentences, with low errors of omission and commission for all complexity levels. Increasing the number of words per part of speech class slows the rate of learning, but improvement is rapid in all conditions and the reduction with more complex grammars is modest. Inspection of the context hierarchy, which encodes word and phrasal classes, showed the system grouped nouns with nouns, verbs with verbs, and so forth, as desired.

In general, both experiments produced results consistent with our expectations. They showed that TRELLIS can acquire accurate grammatical expertise, with low errors of omission and commission, from a moderate number of sentence-parse pairs. The rate of learning was rapid, but increasing the number of nonterminal symbols and lexical items led to modest slowing, suggesting that the system scales reasonably in response to both forms of grammatical complexity. Examination of the content hierarchy showed concepts that mapped well onto nonterminals in the target grammar and inspection of the context hierarchy showed categories for target word classes like determiners, adjectives, nouns, verbs, and prepositions.

Overall, these results indicate that TRELLIS' extension of Cobweb to support linked content and context hierarchies support the rapid acquisition of chunks and concepts, both of which are important in grammar induction. We considered briefly the idea of a lesion study that compared TRELLIS' behavior to that of Cobweb, but rejected it because the earlier system only handles individual items and not collections of related items like words in sentences. The experiments demonstrate clearly that the new system's ability to represent, use, and learn both concepts and chunks give it this ability. Certainly many other systems can acquire grammatical knowledge, but few of them offer such a unified theoretical account of chunks and concepts.

The aggregate scores say that TRELLIS parses and generates well, but not what it has learned. Figure 6 opens the two taxonomies after training on the MED grammar, coloring each node by the syntactic classes of the elements beneath it; the labels are ours for interpreting the structure, not used by the system. Two patterns stand out. First, nodes are close to pure well above the leaves, so a node stands for a class of elements rather than the items that created it. Second, the two taxonomies carve memory along different dimensions: content separates chunks by the classes of their parts, splitting determiner-noun pairs from verb-phrase structures, whereas context groups elements appearing in similar surroundings, placing nouns beside adjective phrases that fill the same slots. This division of labor is what lets a chunk learned over one set of words apply to another.

## 6. Related Research on Concepts and Chunks

Our unified theory offers a promising account of how concepts and chunks are two sides of the same coin and the TRELLIS results suggest its viability, yet other frameworks have addressed the problem and merit discussion. The closest are other variants of Cobweb, starting with LABYRINTH (Thompson & Langley, 1991), which encoded, used, and learned both primitive and composite concepts, the latter similar to chunks. However, it lacked a contextual taxonomy and suffered substantial order effects. Another descendant, Trestle (MacLellan et al., 2016), operated over instances organized in a partonomic hierarchy, but flattened the representation during processing. A convolutional version

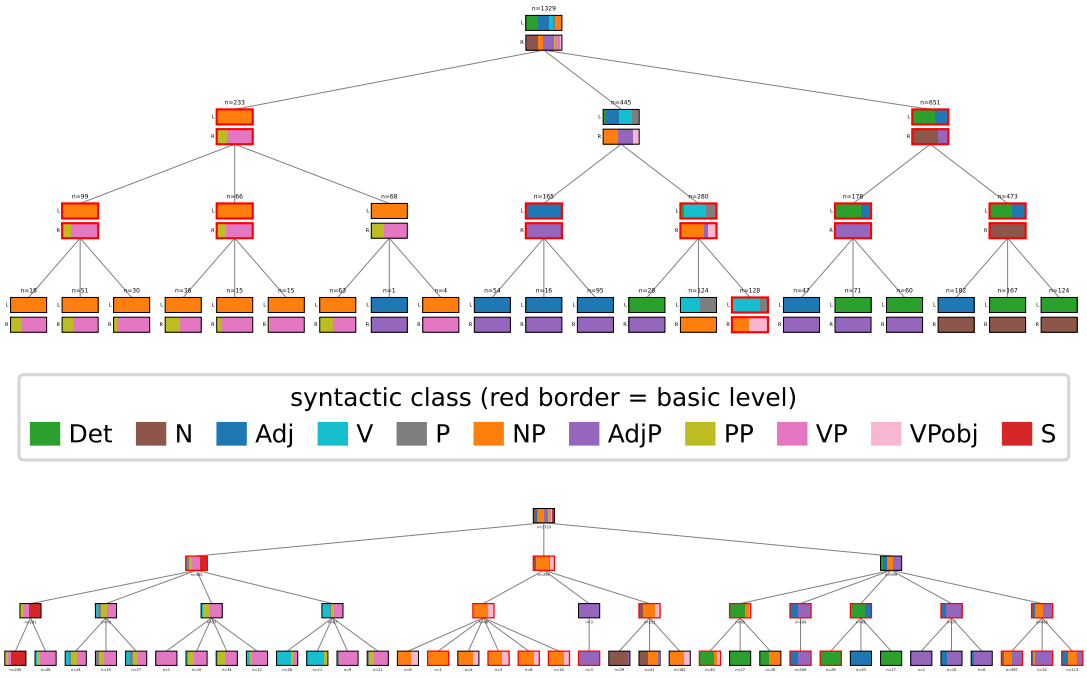



*Figure 6.* Content (top) and context (bottom) taxonomies after training on the MED grammar. Each node shows the distribution of syntactic classes among the elements sorted through it, with $n$ the number of such elements. For content nodes the upper and lower bars give the left and right constituent, respectively. Red borders mark basic-level nodes, and the legend applies to both panels. The labels are only for inspection purposes and play no role in learning. Nodes become nearly pure well above the leaves, which is the generalization that lets one chunk cover many fillers.

of Cobweb (MacLellan & Thakur, 2021) learns over multiple levels of image descriptions, giving some effects of chunks without treating them explicitly.

A second line of Cobweb research treats problem solving as the source of composite structures. Yoo and Fisher's (1991) EXOR system formed concepts over successful problem decompositions, which had forms much like our parse trees, but it generated these from domain rules provided in advance. In addition, EXOR constructed a single taxonomic hierarchy, with each node storing both a chunk-like decomposition and a probabilistic summary used to retrieve it. Other work by Yang and Fisher (1989) and by Langley and Allen (1993) reported systems that used Cobweb-like methods to cluster operators from successful means-ends problem solving, but their taxonomies did not store explicit decompositions. In addition, Fisher and Pazzani (1991) argued that clustering should be guided by the function that categories will serve, which is one way to read our decision to maintain separate content and context taxonomies.

Research on discrimination networks shares many features with the Cobweb family. EPAM (Feigenbaum, 1961; Richman, 1991) organizes knowledge in a univariate taxonomy that it learns with incremental, unsupervised methods, and its operations for extending the network are the same

as Cobweb's but predated them by 25 years. The CHREST architecture (Gobet & Lane, 2012) extends EPAM on multiple fronts, including the addition of lateral links among nodes, similar to the interleaving in TRELLIS. Moreover, the framework has always supported using terminal nodes (e.g., letters) as tests for recognizing higher-level discriminants (e.g., words). But despite similar treatments of categorization, their accounts of chunking follow different paths. In particular, EPAM and CHREST parse structures from the top down, whereas TRELLIS operates from the bottom up.

Although seldom recognized, neural networks (Rumelhart et al., 1986) also offer a unified narrative about concepts and chunks. Nodes in such a network have very different semantics from those in Cobweb and EPAM, but we can still view them as graded concepts that match to a greater or lesser degree. But we can also view them as chunks that combine simpler elements, as demonstrated by visualizations of learned 'features' as weight maps. Taken together, this framework would offer a compelling unification of concepts and chunks if only it included operations for creating new structures, rather than merely updating weights, did not suffer from catastrophic forgetting, and supported more rapid, human-like learning.

A fourth paradigm, with fewer links to cognitive science, involves the construction of context-free grammars from training cases. Classic systems in this paradigm (Langley & Stromsten, 2000; Wolff, 1982, 2026) include operations for inventing nonterminal symbols (creating chunks) and merging them (forming classes). Although most such systems are strongly committed to symbolic representations, the learned structures are organized in what Langley (1996) has called *inference networks* that are similar in spirit to neural networks. Research on predicate invention for inductive logic programming (Muggleton & Buntine, 1988) has championed related ideas for introducing chunk-like terms. Despite many differences from Cobweb and EPAM, these lines of work have been important sources for our ideas about concepts and chunks.

Our theory also bears a resemblance to some frameworks for logical representations of knowledge. For instance, the KL-ONE system (Brachman & Schmolze, 1985) and its descendants organized concept definitions into *structured inheritance networks*.[4] These have much in common with discrimination networks, with each node being a specialization of its parents and a classification process sorting descriptions through the taxonomy. An important difference is that KL-ONE nodes can have multiple parents whose children denote their composition, which TRELLIS addresses with separate hierarchies. Another is that there has been little work on learning structured inheritance networks from experience. However, in many ways the two frameworks are kindred spirits.

## 7. Concluding Remarks

In this paper, we presented a unified computational theory of concepts and chunks. We reviewed the assumptions behind Cobweb, which forms a taxonomy of probabilistic concepts that it uses for classification and prediction. Next we introduced additional postulates that extend the framework to support both categorization of experience and composition into chunks. The extended framework assumes two linked taxonomies, one based on content descriptions and another on contextual ones. We also covered *Trellis*, an implementation of the theory that invokes Cobweb as a subroutine and

4. We should also note that McCarthy's (1993) notion of context specifies logical conditions on meaning, as opposed to the probabilistic contexts in TRELLIS, which are more closely related to distributional semantics.

tested the system on its ability to learn grammatical expertise from sentence-parse pairs. Our theory bridges the longstanding chasm between accounts of concepts and chunks, which are usually treated as disjoint phenomena. Moreover, experimental results with TRELLIS suggest not only that it can acquire syntactic knowledge, which involves both categories and chunks, but does so from a modest number of training sentences and handle recursive structures.

The creation of chunks is a compression device. Once TRELLIS has introduced a concept for some configuration, the system can refer to it with a single symbol, so the context it must carry shrinks from a large list of primitives to a handful of composites. This is how the two taxonomies support each other: the context description of a composite spans the same stretch of experience as the descriptions of its parts, but it occupies one slot rather than many. Contemporary language models take the opposite approach, retaining every token and paying with the quadratic cost of relating them. We do not claim that TRELLIS competes with such systems at their current scale. We do claim that a memory which builds and refers to its own units offers a better way to handle context than retaining the entire past, and the rapid learning in our experiments shows its benefits.

Despite our progress to date, both the theory and its implementation have limits that we should address. Here are some open questions that future work should attempt to answer:

- Can we replace TRELLIS' greedy parser with a more powerful one that reconsiders its choices when they lead it astray, as on garden path sentences?
- Can we extend chunk learning to identify constituents from sentences alone, without providing sample parse trees, like some earlier systems for grammar induction?
- Can TRELLIS represent, use, and acquire not only context-free grammars but also context-sensitive ones, as suggested by its context taxonomy?
- Can the framework learn in other settings that require both generalization and composition, such as understanding scenes, generating plans, and playing games like chess?
- Can TRELLIS acquire syntactic expertise not only from sentences generated by synthetic grammars but also from natural corpora like the Penn Treebank?
- Can the system enable a new variety of large language model that has more interpretable structures, performs more efficiently, and learns more rapidly than existing ones?

We hope to address each of these challenges in future research by continuing to incorporate ideas from earlier efforts and combining them in novel ways, following the tradition established by other researchers in the cognitive systems movement.

## Acknowledgements

The research reported here was supported by Grant No. FA9550-23-1-0580 from the US Air Force Office of Scientific Research, which is not responsible for its contents. We thank Chris MacLellan, Zekun Wang, Xin Lian, Kyle Moore, and friends at the Teachable AI Lab for useful discussions.

## References


Barari, N., Lian, X., & MacLellan, C. J. (2024). Avoiding catastrophic forgetting in visual classification using human concept formation. *Proceedings of the Eleventh Annual Conference on Advances in Cognitive Systems*.

Brachman, R. J., & Schmolze, J. G. (1985). An overview of the KL-ONE knowledge representation system. *Cognitive Science*, *9*, 171–216.

Campbell, M. S., & Berliner, H. J. (1983). A chess program that chunks. *Proceedings of the Third National Conference on Artificial Intelligence* (pp. 49–53). Washington, DC: AAAI Press.

Feigenbaum, E. A. (1961). The simulation of verbal learning behavior. *Proceedings of the Western Joint Computer Conference* (pp. 121–132). Los Angeles, CA.

Fisher, D. H. (1987). Knowledge acquisition via incremental conceptual clustering. *Machine Learning*, *2*, 139–172.

Fisher, D. H. (1989). Noise-tolerant conceptual clustering. *Proceedings of the 11th International Joint Conference on Artificial Intelligence (IJCAI-89)* (pp. 825–830). Detroit, MI: Morgan Kaufmann.

Fisher, D. H., & Pazzani, M. J. (1991). Theory-guided concept formation. In D. H. Fisher, M. J. Pazzani, & P. Langley (Eds.), *Concept formation: Knowledge and experience in unsupervised learning*, 165–178. San Francisco, CA: Morgan Kaufmann.

Fisher, D. H., Pazzani, M. J., & Langley, P. (Eds.). (1991). *Concept formation: Knowledge and experience in unsupervised learning*. San Francisco, CA: Morgan Kaufmann.

Fountain, S. B., & Doyle, K. E. (2012). Learning by chunking. In N. M. Seel (Ed.), *Encyclopedia of the sciences of learning*. New York, NY: Springer.

Gobet, F., & Lane, P. C. R. (2012). CHREST: A model of perception, learning, and problem solving. In N. M. Seel (Ed.), *Encyclopedia of the sciences of learning*, 556–559. New York, NY: Springer.

Gupta, A., Singaravadivelan, K., & Wang, Z. (2025). Hierarchical semantic retrieval with cobweb. From `https://arxiv.org/abs/2510.02539`.

Langley, P. (1996). *Elements of machine learning*. San Francisco, CA: Morgan Kaufmann.

Langley, P. (2025). Concepts and chunks in cognitive systems. *Advances in Cognitive Systems*, *11*, 33–44.

Langley, P., & Allen, J. A. (1993). A unified framework for planning and learning. In S. Minton (Ed.), *Machine learning methods for planning*, 317–349. San Mateo, CA: Morgan Kaufmann.

Langley, P., & Stromsten, S. (2000). Learning context-free grammars with a simplicity bias. *Proceedings of the Eleventh European Conference on Machine Learning* (pp. 220–228). Barcelona, Spain: Springer.

MacLellan, C. J., Harpstead, E., Aleven, V., & Koedinger, K. R. (2016). TRESTLE: A model of concept formation in structured domains. *Advances in Cognitive Systems* (pp. 131–150).

MacLellan, C. J., Matsakis, P., & Langley, P. (2022). Efficient induction of language models via probabilistic concept formation. *Proceedings of the Tenth Annual Conference on Advances in Cognitive Systems*.

MacLellan, C. J., & Thakur, H. (2021). Convolutional Cobweb: A model of incremental learning from 2D images. *Proceedings of the Ninth Annual Conference on Advances in Cognitive Systems*.

McCarthy, J. (1993). Notes on formalizing context. *Proceedings of the Thirteenth International Joint Conference on Artificial Intelligence (IJCAI-93)* (pp. 555–560). Chambéry, France: Morgan Kaufmann.

Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, *63*, 81–97.

Muggleton, S., & Buntine, W. (1988). Machine invention of first-order predicates by inverting resolution. *Proceedings of the Fifth International Conference on Machine Learning* (pp. 339–352). Ann Arbor, MI: Morgan Kaufmann.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, *1*, 81–106.

Richman, H. B. (1991). Discrimination net models of concept formation. In D. H. Fisher, M. J. Pazzani, & P. Langley (Eds.), *Concept formation: Knowledge and experience in unsupervised learning*. San Francisco, CA: Morgan Kaufmann.

Rips, L. J., Smith, E. E., & Medin, D. L. (2012). Concepts and categories: Memory, meaning, and metaphysics. In K. J. Holyoak & R. G. Morrison (Eds.), *The oxford handbook of thinking and reasoning*, 177–209. Oxford, UK: Oxford University Press.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning internal representations by back-propagating errors. In D. E. Rumelhart & J. L. McClelland (Eds.), *Parallel distributed processing*, volume 1, 318–362. Cambridge, MA: MIT Press.

Thompson, K., & Langley, P. (1991). Concept formation in structured domains. In D. H. Fisher, M. J. Pazzani, & P. Langley (Eds.), *Concept formation: Knowledge and experience in unsupervised learning*, 127–161. San Francisco, CA: Morgan Kaufmann.

Wolff, J. G. (1982). Language acquisition, data compression, and generalization. *Language and Communication* (pp. 57–89).

Wolff, J. G. (2026). *Intelligence via compression of information: The sp theory of intelligence and its realisation in the sp computer model*. Singapore: World Scientific Publishing.

Yang, H., & Fisher, D. (1989). Conceptual clustering of means-ends plans. *Proceedings of the Sixth International Workshop on Machine Learning*. Ithaca, NY: Morgan Kaufmann.

Yoo, J., & Fisher, D. (1991). Concept formation over explanations and problem-solving experiences. *Proceedings of the Twelfth International Joint Conference on Artificial Intelligence* (pp. 630–636). Sydney, Australia: Morgan Kaufmann.

## Appendix A. Synthetic Grammars

The first experiment in Section 5 used three synthetic context-free grammars that we describe here. They are strictly nested, so we present them as one set of entries, each tagged with the smallest grammar in which it appears (S = SMALL, M = MED, L = LARGE). Every non-terminal expansion is strictly binary and adjective-free NPs take use the rule NP → Det N, so the grammar never produces a unary syntactic node. The MED grammar adds adjective recursion through an AdjP rule that carries at least one adjective, along with prepositional phrases and direct objects. The LARGE grammar adds relative clauses through Nbar and RelPro nonterminals. Sentence generation samples from alternative expansions for nonterminals, most of which have equal probabilities. Three exceptions keep RelClause sentences near 20% of the LARGE corpus and keep adjective chains short: MED NP splits 3:2 (adjective-free nouns / AdjP); LARGE NP splits 6:2:1 (adjective-free snoun / AdjP / Nbar), AdjP and Nbar each split 2:1; and the AdjP recursion decays as $(1/3)^d$ with depth $d$.

| *Productions* | *Lexicon* |
|---|---|
| S S → NP VP | S Det → the \| a |
| S NP → Det N | L Det → this \| that |
| M NP → Det AdjP | S N → cat \| dog \| man \| woman \| park \| telescope |
| L NP → Det Nbar | L N → boy \| girl \| teacher |
| M AdjP → Adj N \| Adj AdjP | M Adj → big \| small \| red \| quick \| lazy |
| L Nbar → N RelClause \| AdjP RelClause | L Adj → tall \| curious |
| S VP → V NP | S V → saw \| liked \| chased \| found \| admired |
| M VP → V VPobj | L V → carried \| read |
| M VPobj → NP PP | M P → with \| in \| on \| under |
| L RelClause → RelPro VP | L P → near |
| M PP → P NP | L RelPro → who \| that \| which |

The second experiment in Section 5 holds the MED rewrite rules fixed and varies only the lexicon, drawing the first $k$ words from a shared pool per part-of-speech class. The variants are strictly nested, with (LOW having 11 terminals: Det = {the, a}, N = {cat, dog, man}, Adj = {big, small}, V = {saw, liked}, and P = {with, in}. MED has 22 words, adding: N + {woman, park, telescope}, Adj + {red, quick, lazy}, V + {chased, found, admired}, and P + {on, under}. HIGH reaches 39 words by adding: Det + {this, that}, N + {robot, apple, book, girl, boy, bird}, Adj + {ancient, blue, tall, wise}, V + {carried, read, knew}, and P + {near, behind}.

## Appendix B. Implementation Formulas

TRELLIS includes a number of formulae and system-level parameters whose settings influence its performance and learning. We describe each of them briefly below.

**Recognition score.** From postulate R3, the log probability of an instance $\mathbf{i}$ under a concept $C$ is

$$\log P(\mathbf{i} \mid C) = \sum_{(a,v)\in\mathbf{i}} \log P(v \mid C, a), \qquad P(v \mid C, a) = \frac{n_{C,a,v} + \alpha_a}{n_{C,a} + V_a \alpha_a},$$

where $n_{C,a,v}$ counts value $v$ for attribute $a$ at $C$, $V_a$ is the value-vocabulary size, and $\alpha_a$ the smoothing constant. The system ranks candidates by the *class posterior* $\log P(C \mid \mathbf{i}) = \log P(\mathbf{i} \mid C) + \log P(C) - \log P(\mathbf{i})$: normalizing by the instance marginal makes the score discriminative, so non-constituents have low scores. The ranking key sums the two posteriors, $s_c + s_x$.

**Recognition threshold.** Parsing must decide whether a candidate chunk is familiar enough to commit, which TRELLIS determines by asking which level of the taxonomy already knows it. When sorting halts at a node, the system walks upward and selects the first non-root ancestor whose count exceeds $\tau_{\text{parse}}$. Primitives are gated on their context-hierarchy path and composites on their content-hierarchy path, since those are the paths that identify them. In every reported condition we use $\tau_{\text{parse}} = 30$. This strategy keeps TRELLIS from using chunks until they have been encountered at least a few times, increasing their reliability.

**Recall threshold.** Generation must also decide how general a chunk to expand, and TRELLIS answers this with the same upward walk, used twice. First, the system starts at the seed's leaf and climbs to the deepest ancestor with a count above $\tau_{\text{sub}}$, which we set to 20. It then samples a new leaf below that node, giving a different constituent of the same kind. Second, it climbs to the deepest ancestor with a count above $\tau_{\text{gen}}$, which we set to 50, and gathers the chunks stored below it. These form the recombination pool of Section 4.3, which lets the system combine parts that never appeared together in training. The lower threshold keeps a substitution within its type, whereas the higher one widens the pool. Finally, the context hierarchy rejects any chunk that does not fit the class its parent expects.

**Parameter values.** All conditions in our experiments used the same parameter settings. The two smoothing constants $\alpha_{\text{cnt}}$ and $\alpha_{\text{ctx}}$ in the recognition score decide how much probability an attribute holds back for unseen values. We set these to $10^{-4}$ in the content hierarchy and $10^{-5}$ in the context hierarchy, as the smaller value makes context sorting sharper and keeps the two taxonomies apart. Each context instance covers five positions on either side of an element. The nearest neighbor has a weight of one half, and each step further out has half as much again. We set the recognition threshold $\tau_{\text{parse}}$ to 30, the substitution threshold $\tau_{\text{sub}}$ to 20, and the pool threshold $\tau_{\text{gen}}$ to 50.

**Content-instance encoding.** For each child, the content field of Section 4.1 stores not one identifier but a bag of up to three concept identifiers from depth four of the context hierarchy, which preserves the concept's uncertainty rather than committing to its likeliest value. Because the context hierarchy restructures as instances arrive, these references can dangle, so TRELLIS canonicalizes each to a recent ancestor of its target leaf and rewrites stored instances accordingly.